FlexyPool comes with the following default failover strategies:

Table 2.3: FlexyPool failover strategies

Name	Description
Increment pool size on timeout	The connection pool has a *minimum size* and, on demand, it can grow up to its *maximum size*. This strategy increments the target connection pool maximum size on connection acquisition timeout. The *overflow* is a buffer of extra connections allowing the pool to grow beyond its initial *maximum size* until it reaches the *overflow size* threshold
Retrying attempts	This strategy is useful for those connection pools lacking a connection acquiring retry mechanism, and it simply reattempts to fetch a connection for a given number of tries

2.4.1 A real-life connection pool monitoring example

The following example demonstrates how FlexyPool failover strategies can determine the right connection pool size. The application under test is a batch processor using Bitronix transaction manager[5] as the database connection pooling provider.

The batch processor is given a certain data load, and the pool size automatically grows upon detecting a connection acquisition timeout occurrence. The average and the maximum pool size are determined experimentally, without the need of any prior mathematical calculations.

Prior to running the load testing experiment, it is better to know the current application connection pool settings. According to the Bitronix connection pool documentation[6] the default `acquisitionTimeout` (the maximum time a connection request waits before throwing a timeout exception) is 30 seconds.

A connection acquisition timeout threshold of one second is sufficient for the current experiment, allowing the application to react more quickly to a traffic spike and apply a compensating failover strategy.

The initial `maxPoolSize` is set to one connection, and, upon receiving a connection acquisition timeout, it grows until the `maxOverflow` threshold is reached.

The `retryAttempts` value is intentionally set to a reasonably large value because, for a batch processor, dropping a connection request is a much bigger problem than some occasional transaction response time spikes.

[5] https://github.com/bitronix/btm
[6] https://github.com/bitronix/btm/wiki/JDBC-pools-configuration

The experiment starts with the following initial connection pool settings:

Table 2.4: Initial connection pool settings

Name	Value	Description
minPoolSize	0	The pool starts with an initial size of 0
maxPoolSize	1	The pool starts with a maximum size of 1
acquisitionTimeout	1	A connection request waits for 1s before giving up with a timeout exception
maxOverflow	4	The pool can grow up to 5 connections (initial `maxPoolSize` + `maxOverflow`)
retryAttempts	30	If the final maxPoolSize is reached, and there is no connection available, a request will retry 30 times before giving up.

2.4.1.1 Concurrent connection request count metric

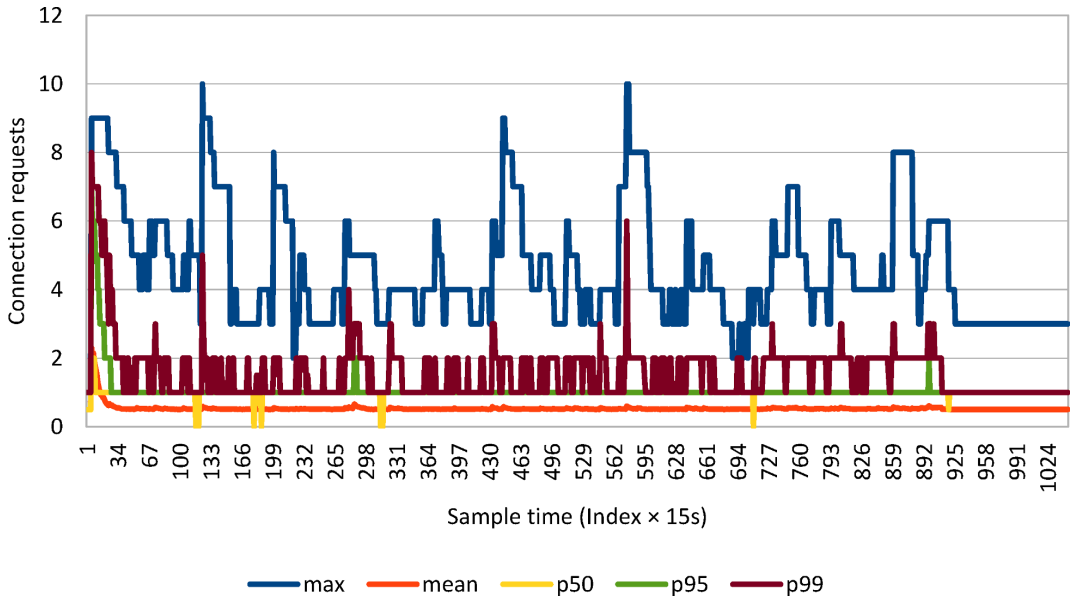

Figure 2.9: Concurrent connection requests

The more incoming concurrent connection requests, the higher the response time (for obtaining a pooled connection) gets. This graph shows the incoming request distribution, making it ideal for spotting traffic spikes.

The average value levels up all outliers, so it cannot reflect the application response to a given traffic spike.

When the recorded values fluctuate dramatically, the average and the maximum value alone offer only a limited view over the actual range of data, and that is why percentiles are preferred in application performance monitoring.

By offering the maximum value, relevant to only a percentage of the whole population, percentiles make outliers visible while capturing the immediate effect of a given traffic change.

2.4.1.2 Concurrent connection count metric

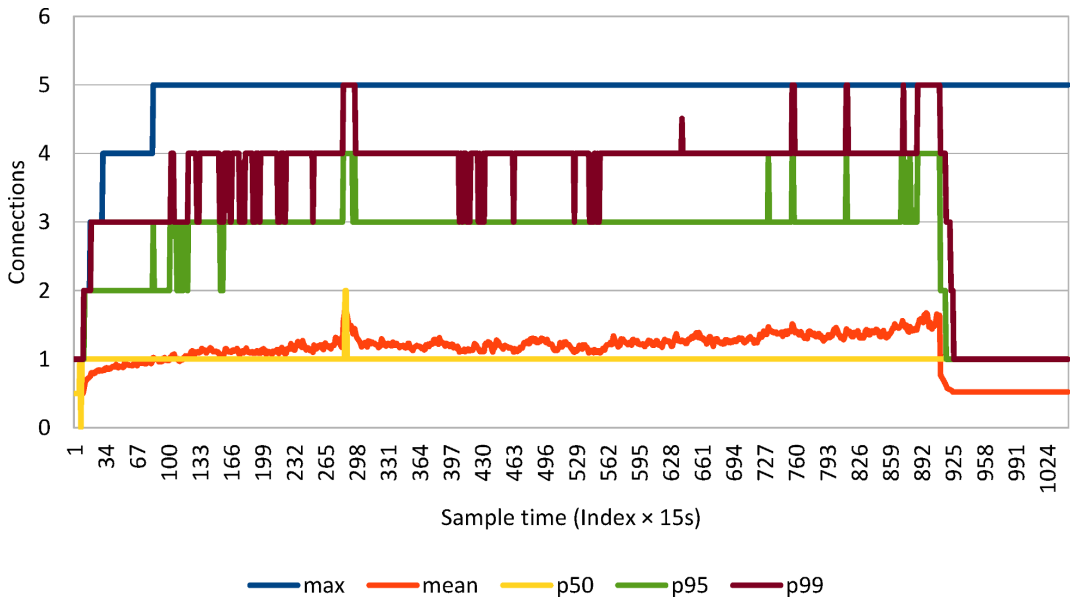

Figure 2.10: Concurrent connections

The average concurrent connection metric follows a gradual slope up to 1.5 connections. Unfortunately, this value is of little use for configuring the right pool size. On the other hand, the 99th percentile is much more informative, showing that 3 to 5 connections are sufficient. The maximum connections graph reconfirms that the pool size should be limited to 5 connections (in case the connection acquisition time is acceptable).

If the connection pool supports it, it is very important to set the idle connection timeout threshold. This way, the pool can release unused connections so the database can provide them to other clients as well.

2.4.1.3 Maximum pool size metric

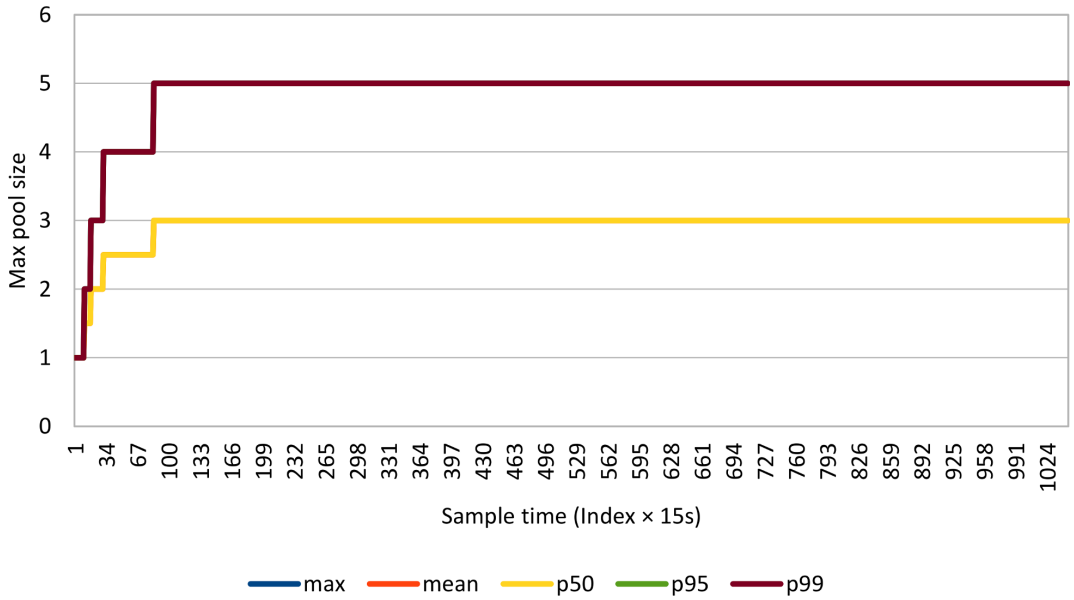

Figure 2.11: Maximum pool size

According to the 99th percentile, the pool gets saturated soon after the job process starts.

2.4.1.4 Connection acquisition time metric

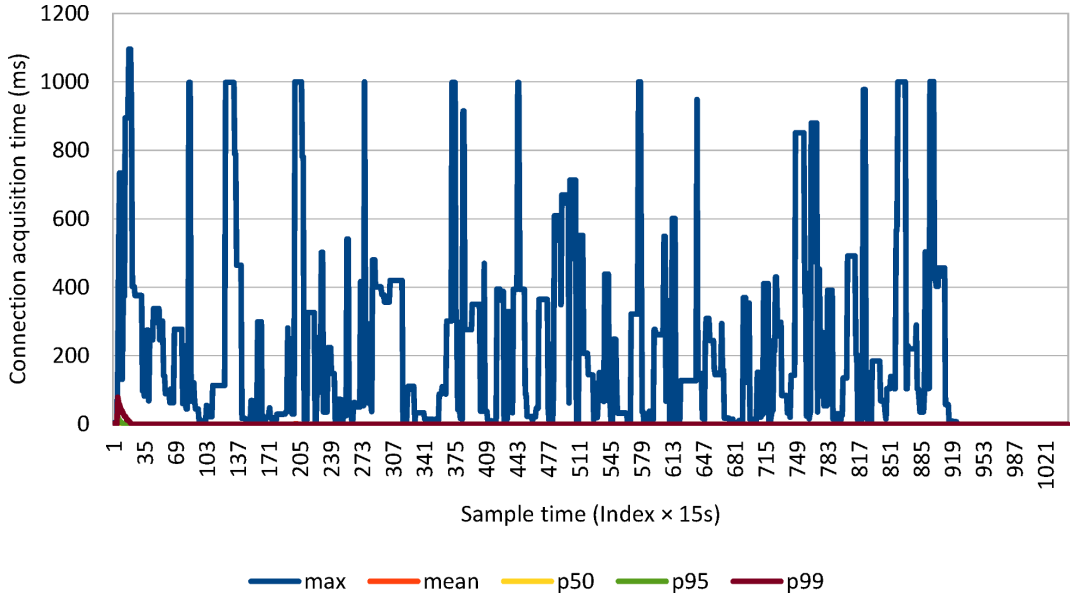

Figure 2.12: Connection acquisition time

The traffic spikes are captured by the maximum graph only. The timeout threshold is hit multiple times as the pool either grows its size or it retries the connection acquisition request.

2.4.1.5 Retry attempts metric

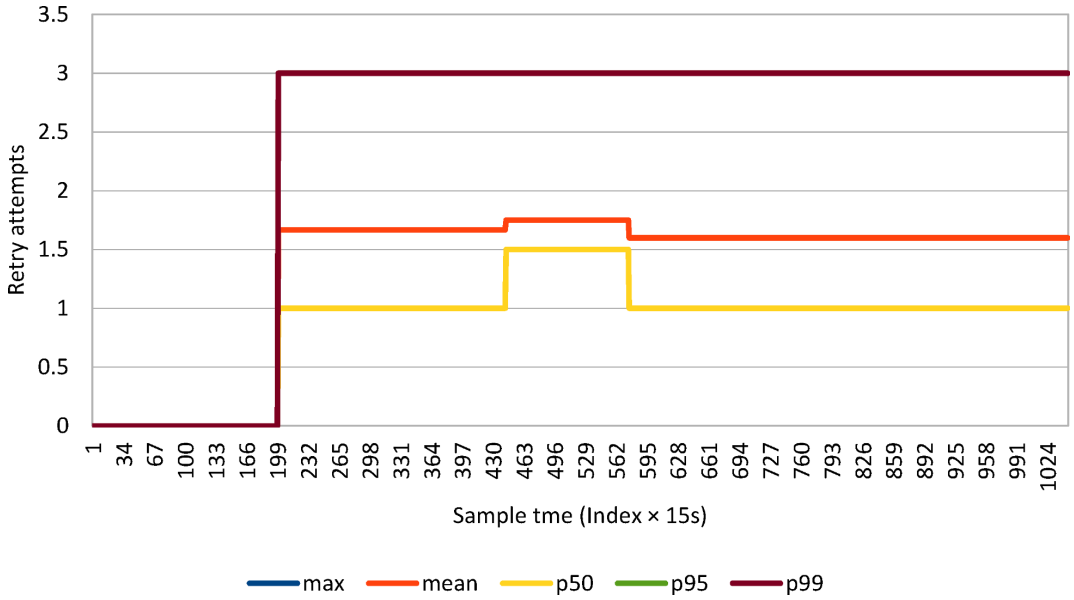

Figure 2.13: Retry attempts

When limiting the connection pool to 5 connections, there are only 3 retry attempts.

2.4.1.6 Overall connection acquisition time metric

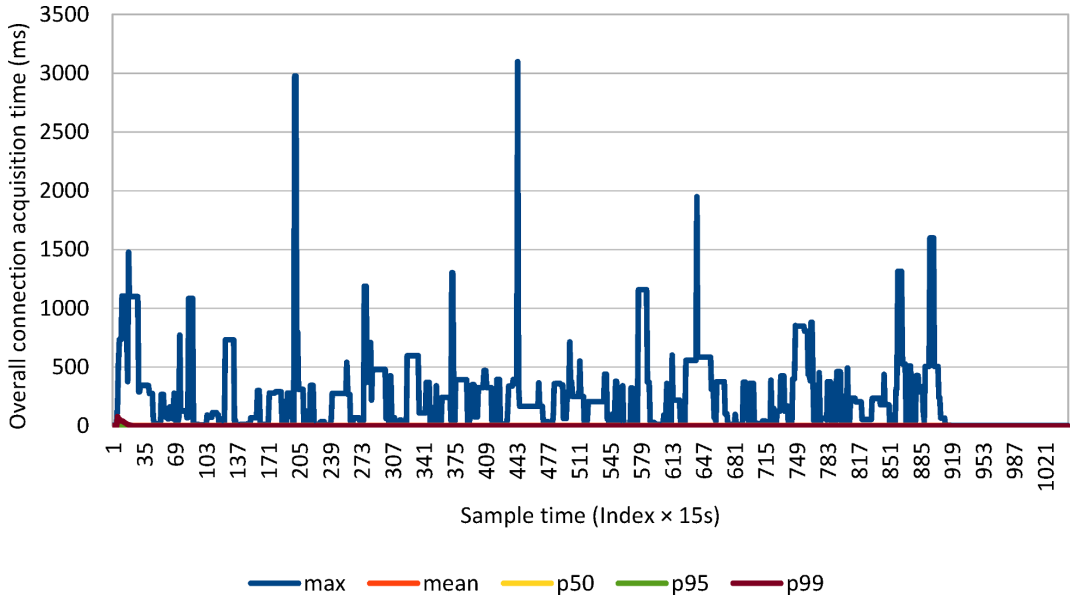

Figure 2.14: Overall connection acquisition time

While the retry attempts graph only shows how the retry count increases with time, the actual effect of reattempting is visible in the overall connection acquisition time.

2.4.1.7 Connection lease time metric

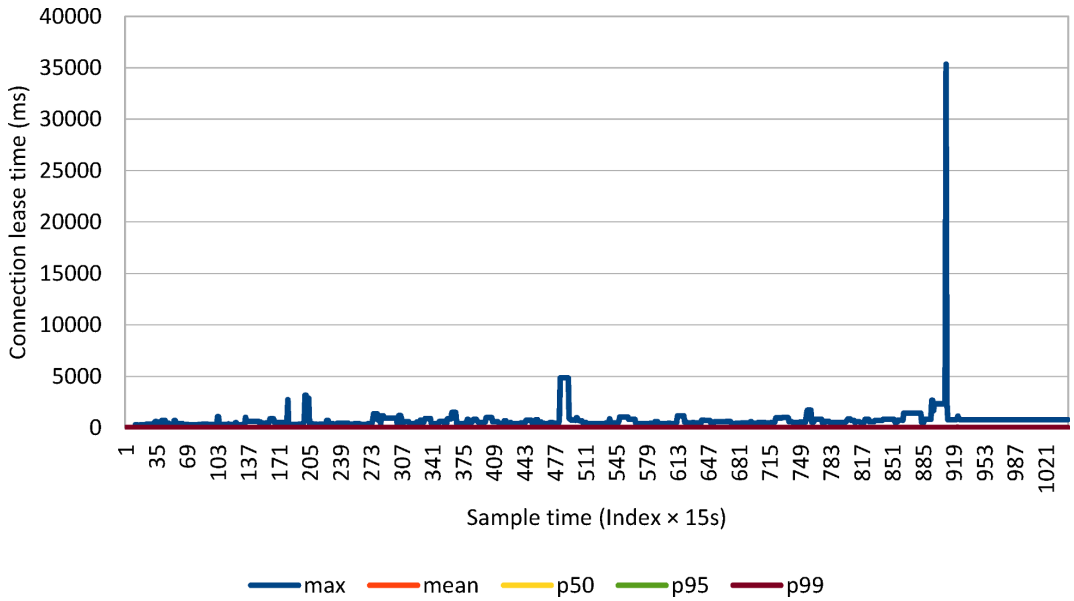

Figure 2.15: Connection lease time

The 99th percentile indicates a rather stable connection lease time throughout the whole job execution. On the other hand, the maximum graph shows a long-running transaction lasting over 35 seconds.

> Holding connections for long periods of time can increase the connection acquisition time, and fewer resources are available to other incoming clients.
>
> Most often, connections are leased for the whole duration of a database transaction. Long-running transactions might hold database locks, which, in turn, might lead to increasing the serial portion of the current execution context, therefore hindering parallelism.
>
> Long-running transactions can be addressed by properly indexing slow queries or by splitting the application-level transaction over multiple database transactions like it is the case in many ETL (Extract, Transform, and Load) systems.

3. Batch Updates

JDBC 2.0 introduced *batch updates* so that multiple DML statements can be grouped into a single database request. Sending multiple statements in a single request reduces the number of database roundtrips, therefore decreasing transaction response time. Even if the reference specification uses the term *updates*, any *insert*, *update* or *delete* statement can be batched, and JDBC supports batching for `java.sql.Statement`, `java.sql.PreparedStatement` and `java.sql.CallableStatement` too.

Not only each database driver is distinct, but even different versions of the same driver might require implementation-specific configurations.

3.1 Batching Statements

For executing static SQL statements, JDBC defines the `Statement` interface, which comes with a batching API as well. Other than for test sake, using a `Statement` for CRUD (Create, Read, Update, Delete), as in the example below, should be avoided for it's prone to SQL injection attacks.

```
statement.addBatch(
    "INSERT INTO post (title, version, id) " +
    "VALUES ('Post no. 1', 0, default)");

statement.addBatch(
    "INSERT INTO post_comment (post_id, review, version, id) " +
    "VALUES (1, 'Post comment 1.1', 0, default)");

int[] updateCounts = statement.executeBatch();
```

The numbers of database rows affected by each statement is included in the return value of the `executeBatch()` method.

Oracle

For `Statement` and `CallableStatement`, the Oracle JDBC Driver[a] does not actually support batching. For anything but `PreparedStatement`, the driver ignores batching, and each statement is executed separately.

[a] http://docs.oracle.com/cd/E11882_01/java.112/e16548/oraperf.htm#JJDBC28752

The following graph depicts how different JDBC drivers behave when varying batch size, the test measuring the time it takes to insert 1000 *post* rows with 4 *comments* each:

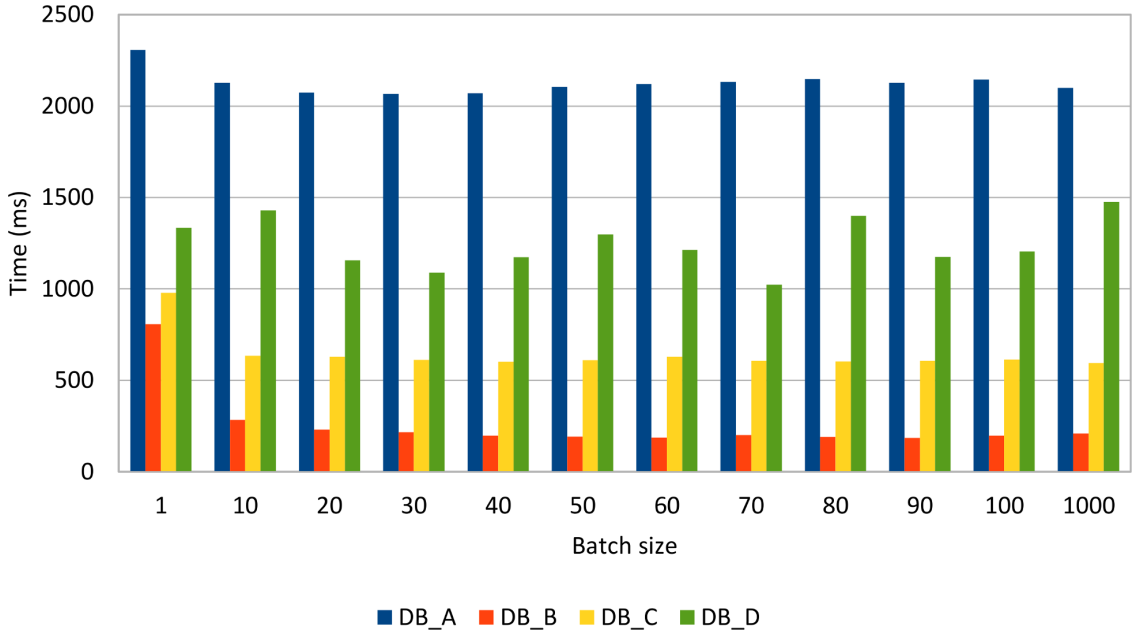

Figure 3.1: Statement batching

Reordering inserts, so that all *posts* are inserted before the *comment* rows, gives the following results:

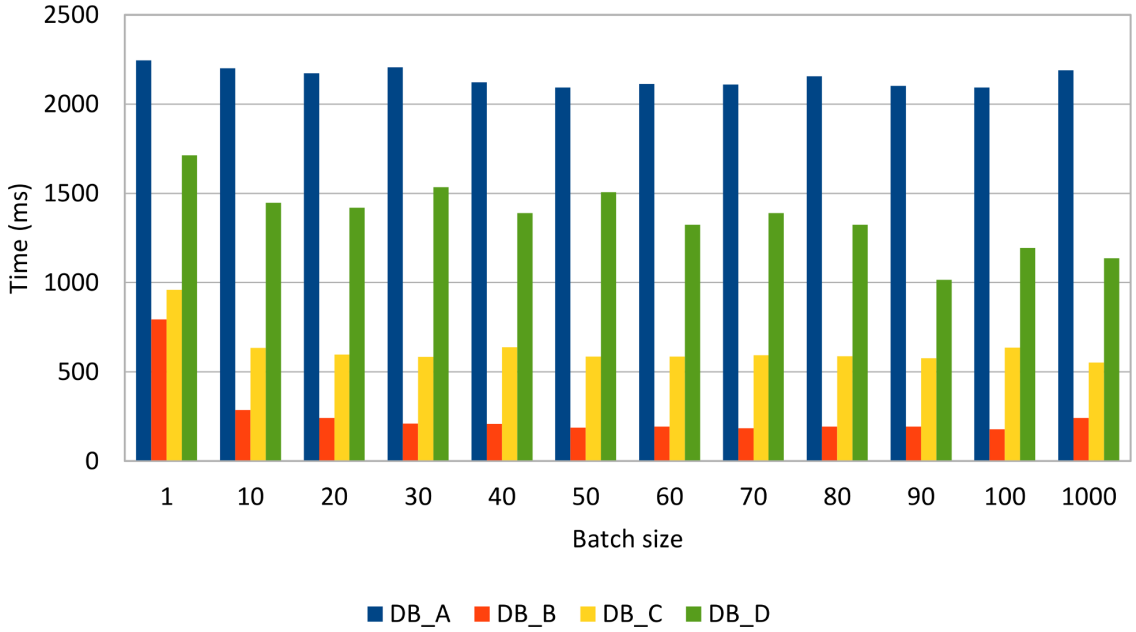

Figure 3.2: Reordered statement batching

Reordering statements does not seem to improve performance noticeably, although some drivers (e.g. MySQL) might take advantage of this optimization.

MySQL

Although it implements the JDBC specification, by default, the MySQL JDBC driver does not send the batched statements in a single request.

For this purpose, the JDBC driver defines the `rewriteBatchedStatements`[a] connection property, so that statements get rewritten into a single `String` buffer. In order to fetch the auto-generated row keys, the batch must contain insert statements only.

For `PreparedStatement`, this property rewrites the batched insert statements into a multi-value insert. Unfortunately, the driver is not able to use server-side prepared statements when enabling rewriting.

Without setting this property, the MySQL driver simply executes each DML statement separately, therefore defeating the purpose of batching.

[a] http://dev.mysql.com/doc/connector-j/en/connector-j-reference-configuration-properties.html

The following graph demonstrates how statement rewriting performs against the default behavior of the MySQL JDBC driver:

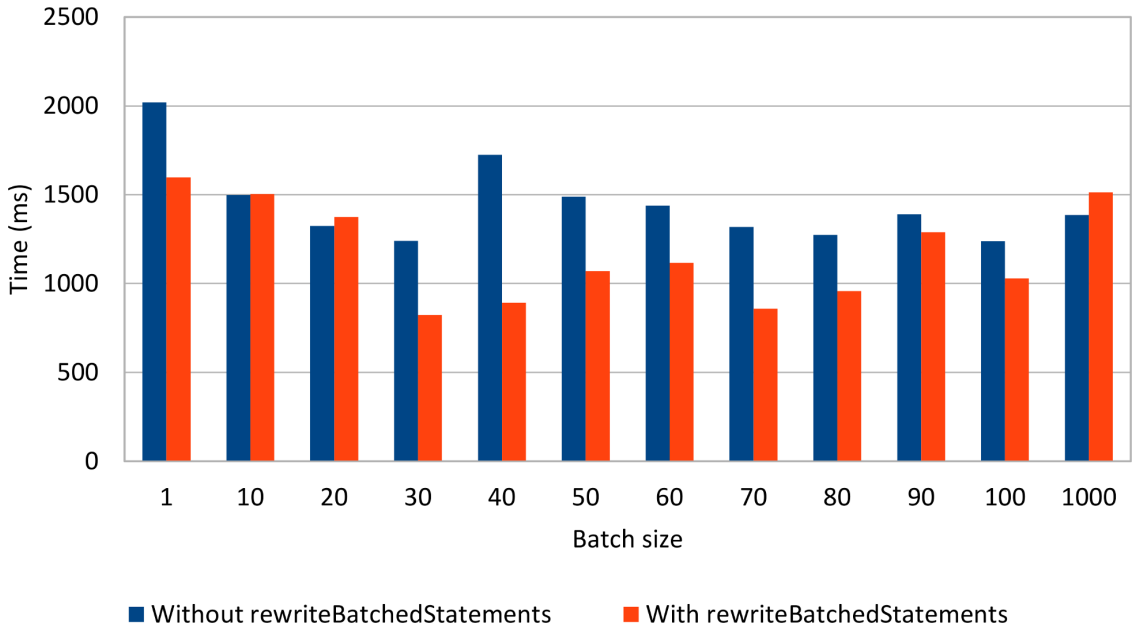

Figure 3.3: MySQL Statement batching

Rewriting non-parameterized statements seems to make a difference, as long as the batch size is not too large. In practice, it is common to use a relatively small batch size, to reduce both the client-side memory footprint and to avoid congesting the server from suddenly processing a huge batch load.

3.2 Batching PreparedStatements

For parameterized statements (a very common enterprise application requirement), the JDBC `Statement` is a poor fit because the only option for varying the executing SQL statement is through `String` manipulation. Using a `String` template or concatenating `String` tokens is risky as it makes the data access logic vulnerable to SQL injection attacks.

To address this shortcoming, JDBC offers the `PreparedStatement` interface for binding parameters in a safe manner. The driver must validate the provided parameter at runtime, therefore discarding unexpected input values.

Because a `PreparedStatement` is associated with a single DML statement, the batch update can group multiple parameter values belonging to the same prepared statement.

```
PreparedStatement postStatement = connection.prepareStatement(
    "INSERT INTO post (title, version, id) " +
    "VALUES (?, ?, ?)");

postStatement.setString(1, String.format("Post no. %1$d", 1));
postStatement.setInt(2, 0);
postStatement.setLong(3, 1);
postStatement.addBatch();

postStatement.setString(1, String.format("Post no. %1$d", 2));
postStatement.setInt(2, 0);
postStatement.setLong(3, 2);
postStatement.addBatch();

int[] updateCounts = postStatement.executeBatch();
```

SQL injection

For an enterprise application, security is a very important technical requirement. The SQL *Injection* attack exploits data access layers that do not use bind parameters. When the SQL statement is the result of `String` concatenation, an attacker could inject a malicious SQL routine that is sent to the database along the current executing statement.

SQL injection is usually done by ending the current statement with the `;` character and continuing it with a rogue SQL command, like modifying the database structure (deleting a table or modifying authorization rights) or even extracting sensitive information.

All DML statements can benefit from batching as the following tests demonstrate. Just like for the JDBC `Statement` test case, the same amount of data (1000 *post* and 4000 *comments*) is inserted, updated, and deleted while varying the batch size.

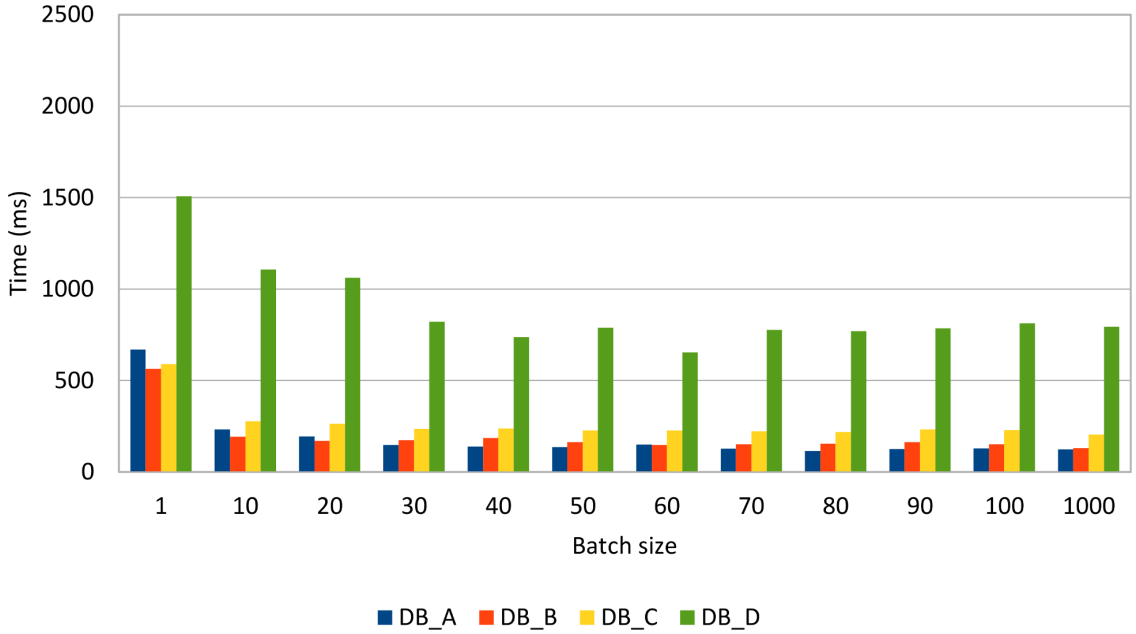

Figure 3.4: Insert PreparedStatement batch size

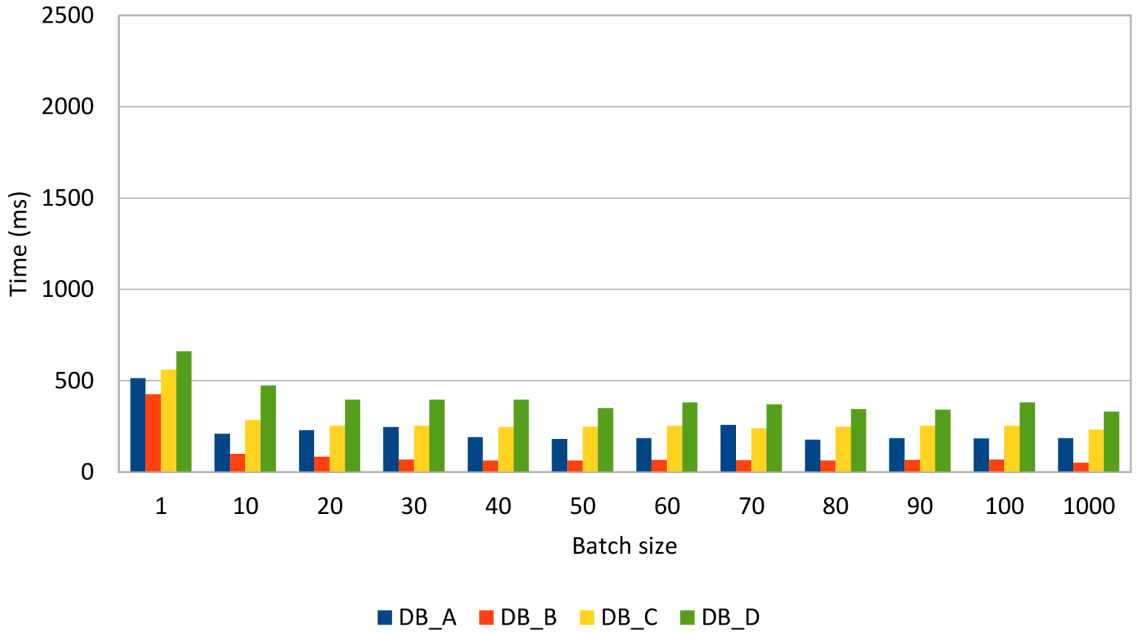

Figure 3.5: Update PreparedStatement batch size

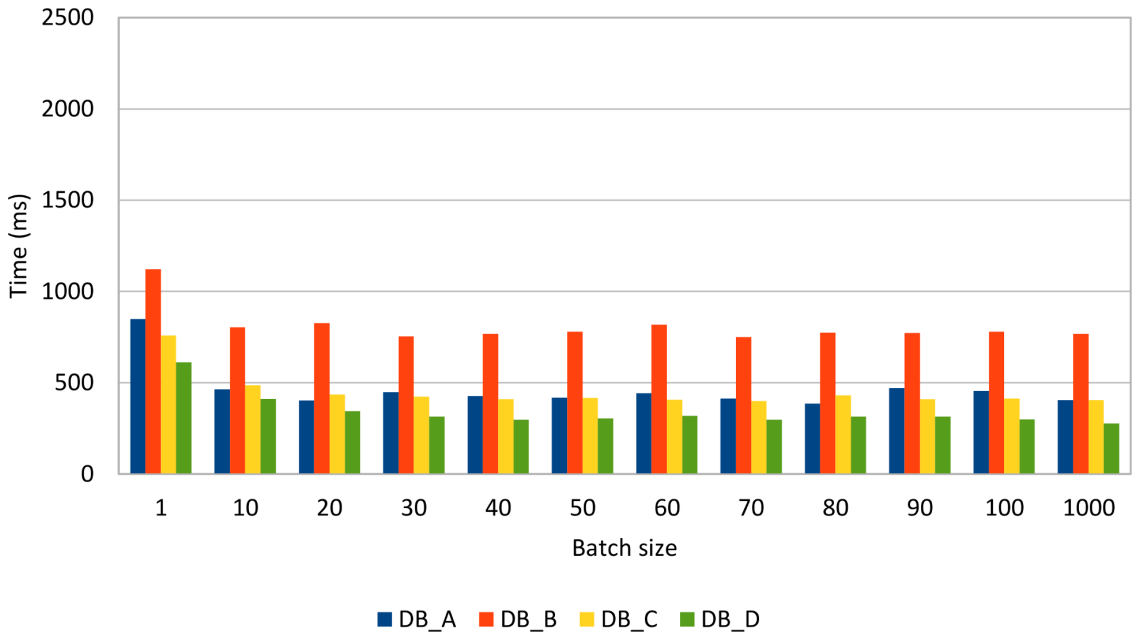

Figure 3.6: Delete PreparedStatement batch size

All database systems show a significant performance improvement when batching prepared statements. Some database systems are very fast when inserting or updating rows while others perform very well when deleting data.

Compared to the previous `Statement` batch insert results, it is clear that, for the same data load, the `PreparedStatement` use case performs just better. In fact, `Statement(s)` should not be used for batching CRUD operations, being more suitable for bulk processing:

```
DELETE from post
WHERE spam = true AND created_on < current_timestamp - INTERVAL '30' day;
```

3.2.1 Choosing the right batch size

Finding the right batch size is not a trivial thing to do as there is no mathematical equation to solve the appropriate batch size for any enterprise application.

Like any other performance optimization technique, measuring the application performance gain in response to a certain batch size value remains the most reliable tuning option.

The astute reader has already figured out that even a low batch size can reduce the transaction response time, and the performance gain does not grow linearly with batch size. Although a larger batch value can save more database roundtrips, the overall performance gain does not necessarily increase linearly. In fact, a very large batch size can hurt application performance if the transaction takes too long to be executed.

As a rule of thumb, you should always measure the performance improvement for various batch sizes. In practice, a relatively low value (between 10 and 30) is usually a good choice.

3.2.2 Bulk processing

Apart from batching, SQL offers bulk operations to modify all rows that satisfy a given filtering criteria. *Bulk update* or *delete* statements can also benefit from indexing, just like select statements.

To update all records from the previous example, one would have to execute the following statements:

```
UPDATE post SET version = version + 1;
UPDATE post_comment SET version = version + 1;
```

Table 3.1: Bulk update time

DB_A time (ms)	DB_B time (ms)	DB_C time (ms)	DB_D time (ms)
26	13	58	9

The bulk alternative is one order of magnitude faster than batch updates. However, batch updates can benefit from application-level optimistic locking mechanisms, which are suitable for preventing *lost updates* when data is loaded in a read-only database transaction and written back in a successive transaction.

Like with updates, bulk deleting is also much faster than deleting in batches.

```
DELETE FROM post_comment WHERE version > 0;
DELETE FROM post WHERE version > 0;
```

Table 3.2: Bulk delete time

DB_A time (ms)	DB_B time (ms)	DB_C time (ms)	DB_D time (ms)
3	12	1	2

Long-running transaction caveats

Processing too much data in a single transaction can degrade application performance, especially in a highly concurrent environment. Whether using 2PL (Two-Phase Locking) or MVCC (Multiversion Concurrency Control), writers always block other conflicting writers.

Long running transactions can affect both batch updates and bulk operations if the current transaction modifies a very large number of records. For this reason, it is more practical to break a large batch processing task into smaller manageable ones that can release locks in a timely fashion.

3.3 Retrieving auto-generated keys

It is common practice to delegate the row identifier generation to the database system. This way, the developer does not have to provide a monotonically incrementing primary key since the database takes care of this upon inserting a new record.

As convenient as this practice may be, it is important to know that auto-generated database identifiers might conflict with the batch insert process.

Like many other database features, setting the auto incremented identifier strategy is database-specific so the choice goes between an *identity* column or a database *sequence* generator.

Oracle

Prior to Oracle 12c, an auto incremented generator had to be implemented on top of a database sequence.

```
CREATE SEQUENCE post_seq;

CREATE TABLE post (
    id NUMBER(19,0) NOT NULL,
    title VARCHAR2(255 CHAR),
    version NUMBER(10,0) NOT NULL,
    PRIMARY KEY (id));

CREATE OR REPLACE TRIGGER post_identity
BEFORE INSERT ON post
FOR EACH ROW
BEGIN
    SELECT post_seq.NEXTVAL
    INTO   :NEW.id
    FROM   dual;
end;
```

Oracle 12c adds support for identity columns as well, so the previous example can be simplified as follows.

```
CREATE TABLE post (
    id NUMBER(19,0) NOT NULL GENERATED ALWAYS AS IDENTITY,
    title VARCHAR2(255 CHAR),
    version NUMBER(10,0) NOT NULL,
    PRIMARY KEY (id));
```

SQL Server

Traditionally, SQL Server offered identity column generators, but, since SQL Server 2012, it now supports database sequences as well.

```
CREATE TABLE post (
    id BIGINT IDENTITY NOT NULL,
    title VARCHAR(255),
    version INT NOT NULL,
    PRIMARY KEY (id));
```

PostgreSQL

PostgreSQL 9.5 does not support identity columns natively, although it offers the `SERIAL` column type which can emulate an identity column.

```
CREATE TABLE post (
    id SERIAL NOT NULL,
    title VARCHAR(255),
    version INT4 NOT NULL,
    PRIMARY KEY (id));
```

The `SERIAL` (4 bytes) and `BIGSERIAL` (8 bytes) types are just a syntactic sugar expression as, behind the scenes, PostgreSQL relies on a database sequence anyway.

The previous definition is therefore equivalent to:

```
CREATE SEQUENCE post_id_seq;

CREATE TABLE post (
    id INTEGER DEFAULT NEXTVAL('post_id_seq') NOT NULL,
    title VARCHAR(255),
    version INT4 NOT NULL,
    PRIMARY KEY (id));
);
```

MySQL

MySQL 5.7 only supports identity columns through the `AUTO_INCREMENT` attribute.

```
CREATE TABLE post (
    id BIGINT NOT NULL AUTO_INCREMENT,
    title VARCHAR(255),
    version INTEGER NOT NULL,
    PRIMARY KEY (id));
```

Many database developers like this approach since the client does not have to care about supplying a database identifier upon inserting a new row.

```
INSERT INTO post (title, version) VALUES (?, ?);
```

To retrieve the newly created row identifier, the JDBC `PreparedStatement` must be instructed to return the auto-generated keys.

```
PreparedStatement postStatement = connection.prepareStatement(
    "INSERT INTO post (title, version) VALUES (?, ?)",
    Statement.RETURN_GENERATED_KEYS
);
```

One alternative is to hint the driver about the column index holding the auto-generated key column.

```
PreparedStatement postStatement = connection.prepareStatement(
    "INSERT INTO post (title, version) VALUES (?, ?)",
    new int[] {1}
);
```

The column name can also be used to instruct the driver about the auto-generated key column.

```
PreparedStatement postStatement = connection.prepareStatement(
    "INSERT INTO post (title, version) VALUES (?, ?)",
    new String[] {"id"}
);
```

It is better to know all these three alternatives because they are not interchangeable on all database systems.

Oracle auto-generated key retrieval gotcha

When using `Statement.RETURN_GENERATED_KEYS`, Oracle returns a `ROWID` instead of the actually generated column value. A workaround is to supply the column index or the column name, and so the auto-generated value can be extracted after executing the statement.

According to the JDBC 4.2 specification, every driver must implement the `supportsGetGeneratedKeys()` method and specify whether it supports auto-generated key retrieval. Unfortunately, this only applies to single statement updates as the specification does not make it mandatory for drivers to support generated key retrieval for batch statements. That being said, not all database systems support fetching auto-generated keys from a batch of statements.

Table 3.3: Driver support for retrieving generated keys

Returns generated keys after calling	Oracle JDBC driver (11.2.0.4)	Oracle JDBC driver (12.1.0.1)	SQL Server JDBC driver (4.2)	PostgreSQL JDBC driver (9.4-1201-jdbc41)	MySQL JDBC driver (5.1.36)
`executeUpdate()`	Yes	Yes	Yes	Yes	Yes
`executeBatch()`	No	Yes	No	Yes	Yes

If the Oracle JDBC driver 11.2.0.4 cannot retrieve auto-generated batch keys, the 12.1.0.1 version works just fine. When trying to get the auto-generated batch keys, the SQL Server JDBC driver throws this exception: *The statement must be executed before any results can be obtained.*

3.3.1 Sequences to the rescue

As opposed to identity columns, database sequences offer the advantage of decoupling the identifier generation from the actual row insert. To make use of batch inserts, the identifier must be fetched prior to setting the insert statement parameter values.

```
private long getNextSequenceValue(Connection connection)
    throws SQLException {
    try(Statement statement = connection.createStatement()) {
        try(ResultSet resultSet = statement.executeQuery(
            callSequenceSyntax())) {
            resultSet.next();
            return resultSet.getLong(1);
        }
    }
}
```

For calling a sequence, every database offers a specific syntax:

Oracle

```
SELECT post_seq.NEXTVAL FROM dual;
```

SQL Server

```
SELECT NEXT VALUE FOR post_seq;
```

PostgreSQL

```
SELECT NEXTVAL('post_seq');
```

Because the primary key is generated up-front, there is no need to call the `getGeneratedKeys()` method, and so batch inserts are not driver dependent anymore.

```
try(PreparedStatement postStatement = connection.prepareStatement(
    "INSERT INTO post (id, title, version) VALUES (?, ?, ?)")) {
    for (int i = 0; i < postCount; i++) {
        if(i > 0 && i % batchSize == 0) {
            postStatement.executeBatch();
        }
        postStatement.setLong(1, getNextSequenceValue(connection));
        postStatement.setString(2, String.format("Post no. %1$d", i));
        postStatement.setInt(3, 0);
        postStatement.addBatch();
    }
    postStatement.executeBatch();
}
```

Many database engines use sequence number generation optimizations to lower the sequence call execution as much as possible. If the number of inserted records is relatively low, then the sequence call overhead (extra database roundtrips) will be insignificant. However, for batch processors inserting large amounts of data, the extra sequence calls can add up.

Optimizing sequence calls

The data access layer does not need to go to the database to fetch a unique identifier if the sequence incrementation step is greater than 1. For a step of N, the sequence numbers are 1, N + 1, 2N + 1, 3N + 1, etc. The data access logic can assign identifiers in-between the database sequence calls (e.g. 2, 3, 4, ..., N -1, N), and so it can mitigate the extra network roundtrips penalty.

This strategy is going to be discussed in greater detail in the Hibernate types and identifiers chapter.

4. Statement Caching

Being a declarative language, SQL describes the *what* and not the *how*. The actual database structures and the algorithms used for fetching and preparing the desired result set are hidden away from the database client, which only has to focus on properly defining the SQL statement. This way, to deliver the most efficient data access plan, the database can attempt various execution strategies.

4.1 Statement lifecycle

The main database modules responsible for processing a SQL statement are the *Parser*, the *Optimizer*, and the *Executor*.

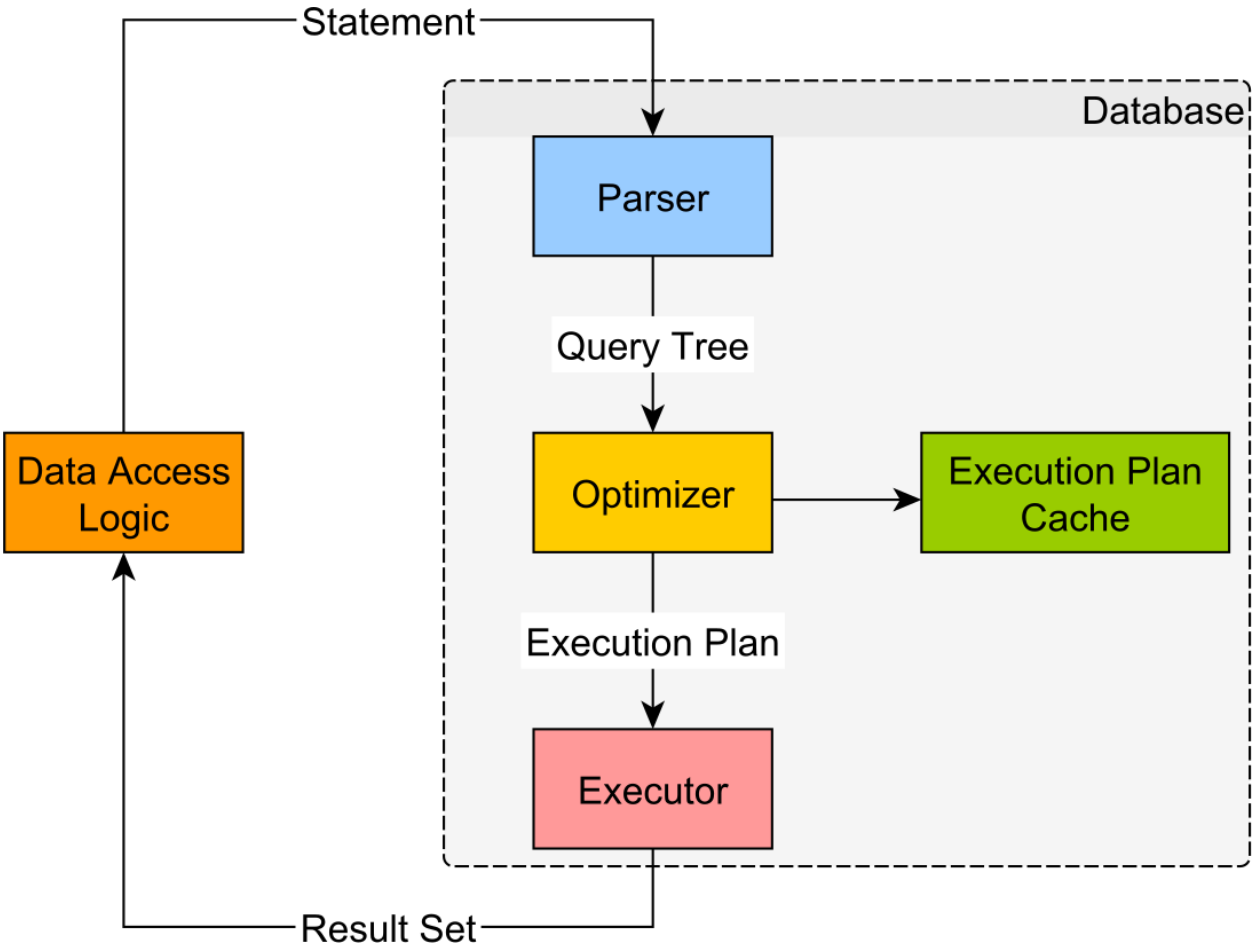

Figure 4.1: Statement lifecycle